

What we have seen so far

Simplecpp vs C++

<code>#include <simplecpp></code>	<code>#include <iostream></code> <code>#include <cmath></code>
<code>main_program{ }</code>	<code>int main(){ }</code>
<code>repeat(n)</code>	<code>for(int i=0; i<n; i++)</code>
<code>cin, cout, endl</code>	<code>std::cin, std::cout, std::endl</code>
<code>cin, cout, endl</code>	<code>cin, cout, endl</code> if you write in the beginning <code>using namespace std;</code>
<code>s++ nim.cpp</code>	<code>g++ nim.cpp</code>

Function declarations

- Function declarations, header files, -c for object files
- Common for both simplecpp and C++
- Function declarations are put at the top. They are useful for
 1. Calling a function above its definition
 2. Calling a function which is in another file.
- Header files are used write the function declarations.
 - Header files are written by the function creator.
 - Convenient for the user, who can include the header file instead of writing all the declarations.

Concept 1: Variables and data types

- There are different types for different purposes like int, char, double, float.
- Mixing them with each other may work some times, can cause error / unexpected behaviour some times. Be careful.
- Each data type has a size in bits, and thus, have a range.
- E.g., int are arranged circularly, starting from -2^{31} to $2^{31} - 1$
- `char x = 'a'` (single quotes)
- `if( x == 'a')`

Concept 1: Variables and data types

- Think of variable **x** as some space allocated in the memory with label **x**
- Thus, can be assigned new values.
- Variables have a scope — identified by {}
- Variables defined in a scope are not accessible outside the scope.
- Scopes can be nested — for example, you can think of them like countries, states, districts.
- A variable defined in MH is not accessible in MP, but accessible inside all districts of MH.

Exercise

- Will this give an error ?

```
int i = 1;
for(int i=0; i<10; i++){
    int x = i*10;
    cout << x << endl;
}
cout << i << endl;
//cout << x << endl;
```


Concept 2: expressions

- $2+3 - 7*2$
- 'f' - 'a'
 - 5
- `char ('F' - 'f' + 'e')`
 - E
- Whereve an int is expected, int expression can be given.
- Boolean expression with operators `&&`, `||`, `!`

Concept 3: Conditional statements

- Essential for writing interesting programs
- `if (condition) { body }`
- `if () {} else {}`
- `if () {} else if () {} else {}`
- Else does not require a condition.
- Else should always accompany an if
- Ternary operator (Booelan expression) ? `expr1` : `expr2`
- Common mistake — `if(x = 0)`

Exercise

- What should 'A' be so that the output of the below program is "Manchester is Red !!"

```
bool A = _____ ;
bool B = true, C = false;
if ( ! A || (A && C) || ! B ) {
    cout << "Manchester is Blue !!" << endl;
}
else {
    cout << "Manchester is Red !!" << endl;
}
```

Concept 3: loops

- `while (condition) {body}`
 - Keep executing the body till condition is true.
- `for(initialization; condition; update) {body}`
 - Initialization is executed before the start of loop
 - Condition is checked before each iteration.
 - Update is made at the end of each iteration.
- `break; //` breaks out of the loop.
- `continue; //` ignores the remaining body, and goes back to condition check.

Exercise

- Output ?

0 0

0 1

1 0

1 1

2 0

2 1

- ```
for(int i=0;i<3;i++){
 for(int j=0; j<3; j++){
 cout << i << " " << j << endl;
 if(j==1) break;
 }
}
```



# Exercise

- Output ?

```
for(int i=0;i<3;i++){
 for(int j=0; j<3; j++){
 if(j==1) continue;
 cout << i << " " << j << endl;
 }
}
```

0 0

0 2

1 0

1 2

2 0

2 2

# Exercise

- Output ?

```
int j=0;
while(1){
 cout << j << endl;
 if (j>20) continue;
 j=j+2;
}
```

# Exercise

- Output ?

```
void modifyValue(int &num) {
 num = num + 10;
}
int main() {
 int value = 5;
 cout << "Before function call: " << value << endl;
 modifyValue(value);
 cout << "After function call: " << value << endl;
 return 0;
}
```